

Hadoop YARN

源码架构分析文档

核心流程图 + 关键类设计说明

基于 Hadoop 3.6.0-SNAPSHOT (release-3.3.5-RC0)

版本: v1.0

目录

第一章 YARN 整体架构概述

- 1.1 模块全景
- 1.2 整体架构图
- 1.3 核心设计模式

第二章 核心流程图

- 2.1 应用提交与运行流程
- 2.2 RMAppl 应用状态机
- 2.3 RMApplAttempt 尝试状态机
- 2.4 NM Container 容器状态机
- 2.5 RMContainer 容器状态机
- 2.6 RMNode 节点状态机
- 2.7 CapacityScheduler 资源分配

第三章 ResourceManager 核心类设计

- 3.1 ResourceManager 主入口
- 3.2 ClientRMService
- 3.3 RMApplManager
- 3.4 RMApplImpl
- 3.5 RMNodeImpl

第四章 调度器设计

- 4.1 调度器继承体系
- 4.2 CapacityScheduler
- 4.3 FairScheduler
- 4.4 FifoScheduler

第五章 NodeManager 核心类设计

- 5.1 NodeManager 主入口
- 5.2 ContainerManagerImpl
- 5.3 ContainerLaunch
- 5.4 ContainerImpl
- 5.5 NodeStatusUpdater

第六章 协议与客户端 API

- 6.1 四大协议接口
- 6.2 YarnClient
- 6.3 AMRMClient
- 6.4 NMClient

第一章 YARN 整体架构概述

1.1 模块全景

Hadoop YARN (Yet Another Resource Negotiator) 是 Hadoop 的资源管理和调度平台。整个 YARN 项目由以下核心模块组成：

模块	说明	文件数
hadoop-yarn-api	协议接口、数据记录(records)、配置常量	272
hadoop-yarn-common	事件框架(AsyncDispatcher)、状态机(StateMachineFactory)、RPC、安全机制	500+
hadoop-yarn-server	ResourceManager、NodeManager、Router、Timeline Service 等14个子模块	1500+
hadoop-yarn-client	客户端SDK (YarnClient/AMRMClient/NMClient) 与CLI工具	43
hadoop-yarn-applications	示例应用 (DistributedShell) 和 YARN Services 框架	100+
hadoop-yarn-registry	基于ZooKeeper的服务发现和注册	70+

表1-1: YARN 核心模块一览

1.2 整体架构图

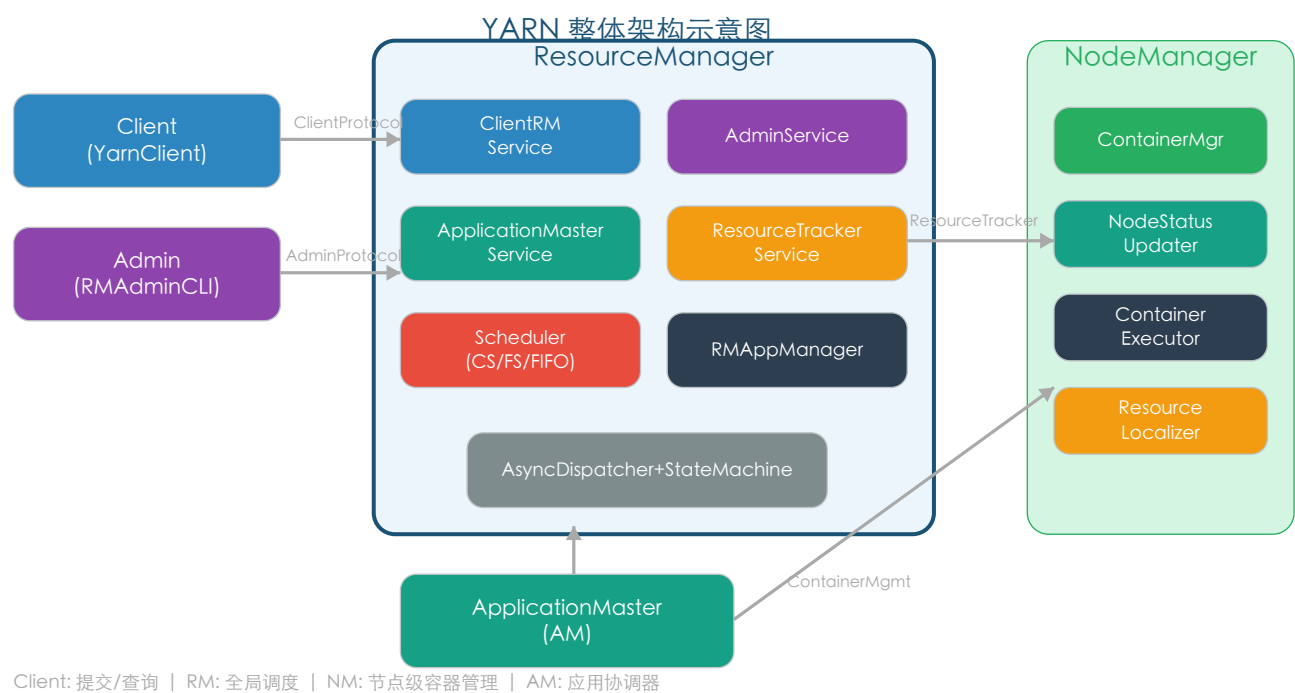


图1-1: YARN 整体架构示意图

1.3 核心设计模式

YARN 采用 事件驱动 + 有限状态机 的核心设计模式。所有关键组件均通过 StateMachineFactory 定义状态转换, 由 AsyncDispatcher 异步分发事件, 实现组件间的松耦合：

设计模式	实现类	说明
事件驱动	AsyncDispatcher, EventHandler	所有组件通过事件解耦, 不同事件类型注册不同Handler
状态机	StateMachineFactory	RMApp/RMAppAttempt/RMNode/Container 均采用FSM管理
服务框架	CompositeService	统一init/start/stop生命周期, 支持服务组合
双层服务	Always-On + Active-Only	支持HA主备切换, Active服务仅在主RM运行
读写锁	ReentrantReadWriteLock	核心类均使用读写锁分离, 保护并发访问

表1-2: YARN 核心设计模式

第二章 核心流程图

2.1 应用提交与运行流程

一个 YARN 应用从提交到运行完成, 需要经过16个关键步骤, 涉及 Client、ResourceManager、NodeManager、ApplicationMaster 四方交互:

YARN 应用提交与运行完整流程

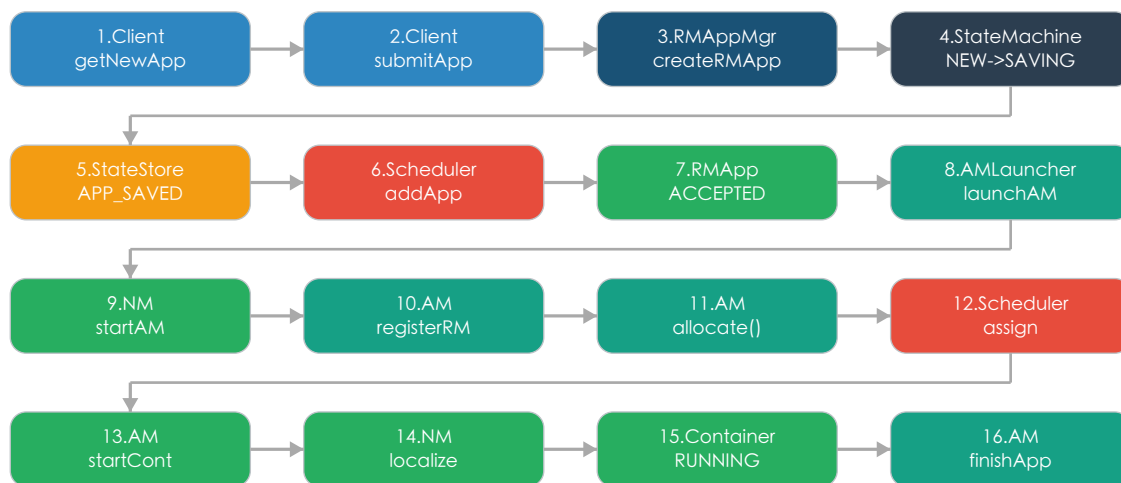


图2-1: 应用提交与运行完整流程

流程说明: Client 先获取 ApplicationId, 然后提交应用; RM 创建 RMAp 状态机并持久化到 StateStore; Scheduler 接受应用后创建 AppAttempt; AMLauncher 在某个 NM 上启动 AM 容器; AM 向 RM 注册并通过 allocate() 心跳申请资源; 获得容器后 AM 通过 ContainerManagementProtocol 在 NM 上启动任务容器; 任务完成后 AM 调用 finishApplicationMaster() 注销。

2.2 RMAp 应用状态机

RMApImpl 定义了11个状态和16种事件类型, 状态机由 StateMachineFactory 构建:

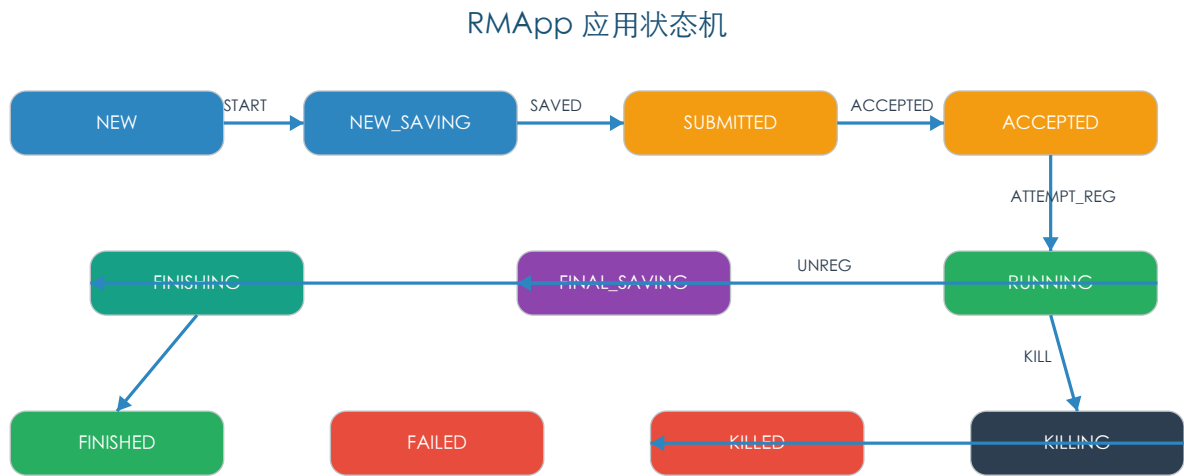


图2-2: RMAp 应用状态机

源状态	事件	目标状态	处理类
NEW	START	NEW_SAVING	RMApNewlySavingTransition
NEW_SAVING	APP_NEW_SAVED	SUBMITTED	AddApplicationToSchedulerTransition
SUBMITTED	APP_ACCEPTED	ACCEPTED	StartAppAttemptTransition
ACCEPTED	ATTEMPT_REGISTERED	RUNNING	RMApStateUpdateTransition
RUNNING	ATTEMPT_UNREGISTERED	FINAL_SAVING	FinalSavingTransition
ACCEPTED/RUNNING	ATTEMPT_FAILED	ACCEPTED/FINAL_SAVING	AttemptFailedTransition(判断重试)
ACCEPTED/RUNNING	KILL	KILLING	KillAttemptTransition

表2-1: RMAp 关键状态转换

2.3 RMApAttempt 尝试状态机

每个 RMAp 可以有多个尝试(Attempt)。RMApAttemptImpl 定义了12个状态, 管理从 AM 容器分配、启动到注册、运行的完整生命周期:

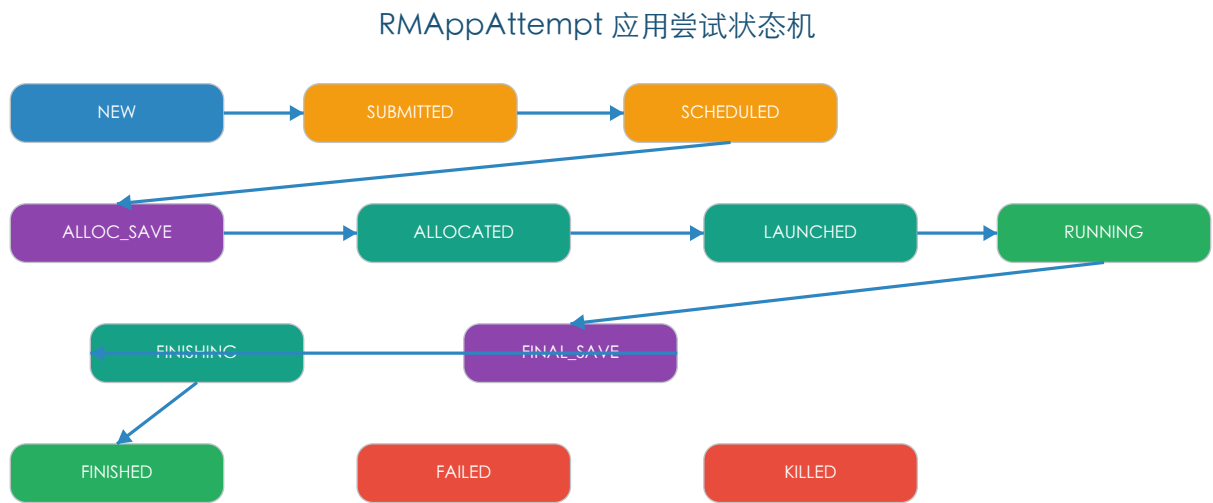


图2-3: RMApAttempt 应用尝试状态机

关键机制: SCHEDULED 状态等待 Scheduler 分配 AM 容器; ALLOCATED_SAVING 先持久化到 StateStore 再真正分配; LAUNCHED 状态等待 AM 注册(有 AMLivelinessMonitor 超时监控); 非托管 AM 直接跳过容器分配进入 LAUNCHED_UNMANAGED_SAVING 路径。

2.4 NM Container 容器状态机

NodeManager 端的 ContainerImpl 定义了17个状态, 管理容器从初始化、资源本地化、调度、运行到退出的完整生命周期:

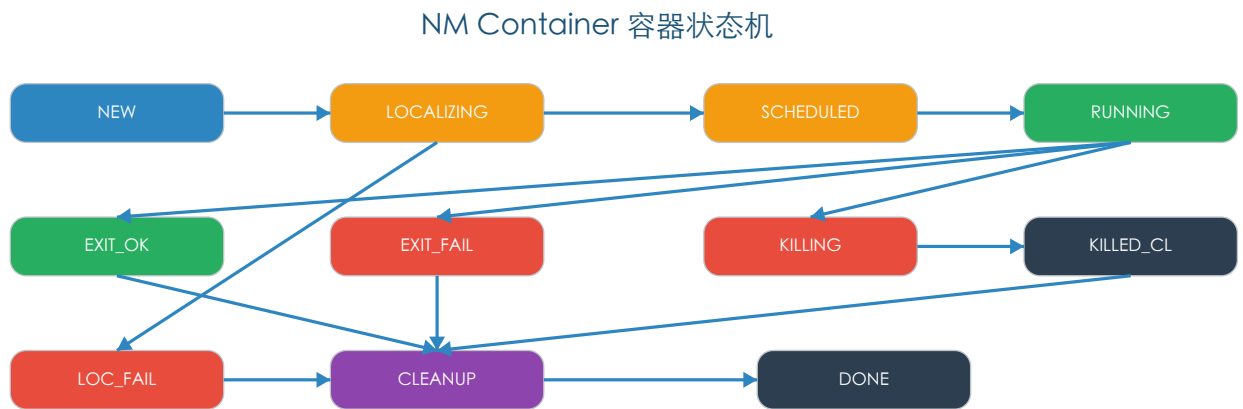


图2-4: NM Container 容器状态机(简化)

核心路径: NEW -> LOCALIZING -> SCHEDULED -> RUNNING -> EXITED_WITH_SUCCESS -> CLEANUP -> DONE。KILL 事件可在任何运行中状态触发。支持重试策略 (ContainerRetryContext)。

2.5 RMContainer (RM侧) 容器状态机

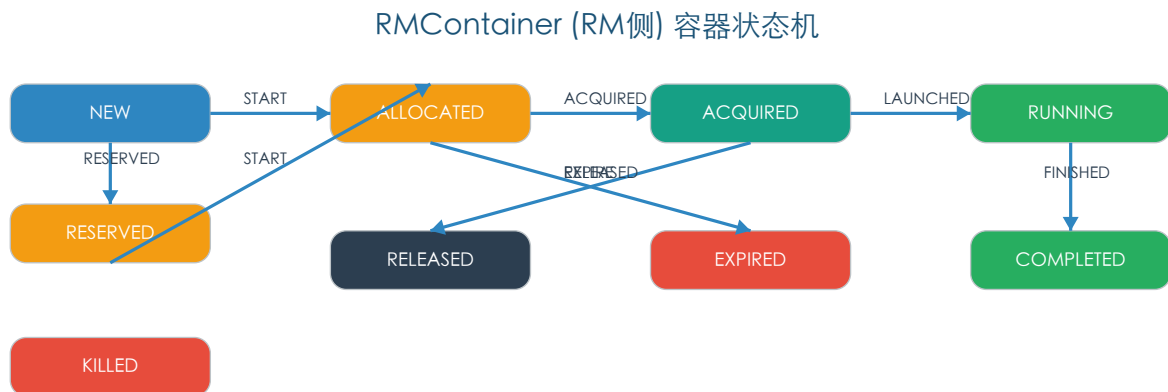


图2-5: RMContainer RM侧容器状态机

说明: RMContainer 是 RM 视角的容器抽象。NEW -> ALLOCATED -> ACQUIRED -> RUNNING -> COMPLETED 为主线; 支持 RESERVED(预留) 路径; 未被及时拉取的容器会 EXPIRE。

2.6 RMNode 节点状态机

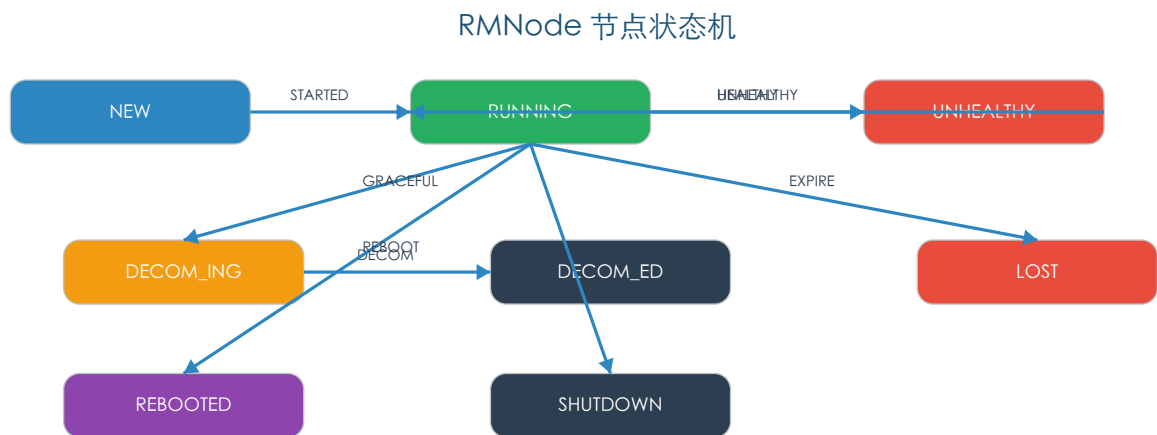


图2-6: RMNode 节点状态机

说明： 节点注册后进入 RUNNING/UNHEALTHY； 心跳维持 RUNNING； 超时进入 LOST； 支持优雅退役(GRACEFUL_DECOMMISSION) 和立即退役(DECOMMISSION)； UNHEALTHY 恢复健康自动回到 RUNNING。

2.7 CapacityScheduler 资源分配流程

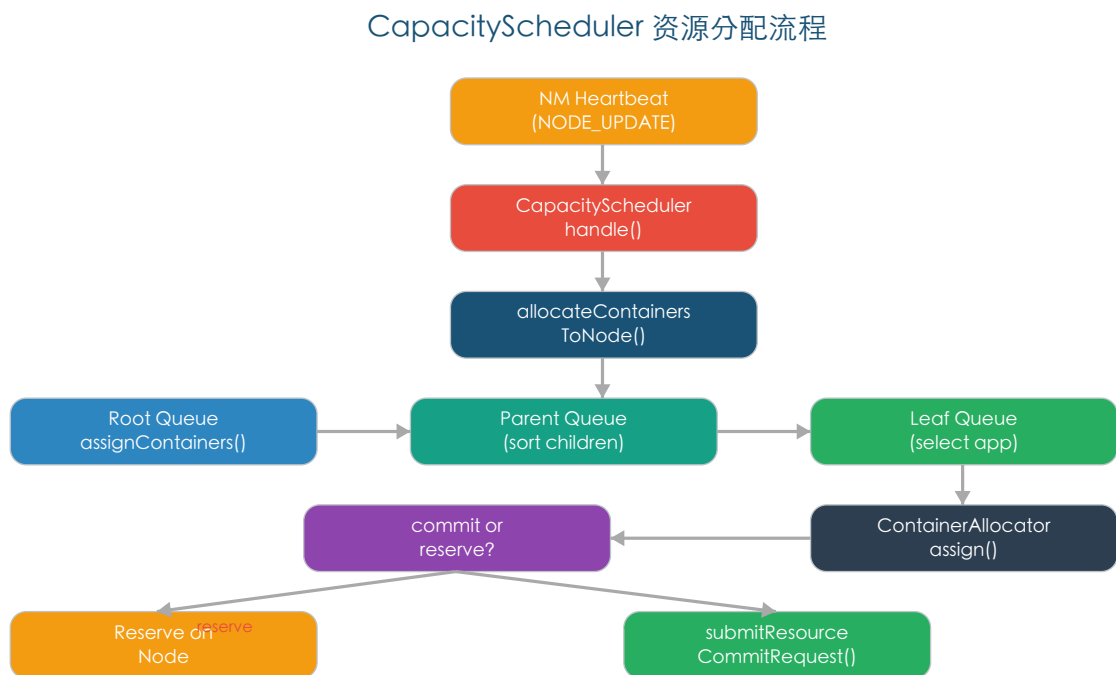


图2-7: CapacityScheduler 资源分配流程

分配算法：NM 心跳触发 NODE_UPDATE；从 Root 队列开始按子队列已用容量比排序，自顶向下递归 assignContainers()；Leaf Queue 选择应用，ContainerAllocator 执行实际分配；资源不足则 Reserve；分配结果通过 submitResourceCommitRequest() 提交。

第三章 ResourceManager 核心类设计

3.1 ResourceManager 主入口

ResourceManager 继承 CompositeService，是 YARN 集群的全局协调者。采用双层服务设计：Always-On 层和 Active 层：

组件	层次	职责
rmDispatcher	Always-On	全局事件分发器, 注册各类EventHandler
adminService	Always-On	处理管理员命令(刷新队列/节点等)
clientRM	Active	处理客户端RPC(提交/查询/杀死应用)
masterService	Active	处理AM心跳和资源分配请求
resourceTracker	Active	处理NM注册和心跳
scheduler	Active	资源调度(CapacityScheduler/FairScheduler)
rmAppManager	Active	应用生命周期管理(提交/完成/清理)
nodesListManager	Active	节点白名单/黑名单管理
nmLivelinessMonitor	Active	NM存活检测(心跳超时处理)

表3-1: ResourceManager 核心组件

初始化流程 (serviceInit): 创建 RMContextImpl -> 安全登录 -> 加载配置 -> 创建 AsyncDispatcher -> 创建 AdminService -> HA选举(如启用) -> 创建 Active 服务集 -> 注册各事件类型的 Handler。

3.2 ClientRMService

实现 ApplicationClientProtocol 接口, 是客户端与 RM 通信的 RPC 服务端：

RPC方法	功能	内部调用
getNewApplication()	生成新的ApplicationId	applicationCounter.incrementAndGet()
submitApplication()	提交应用	rmAppManager.submitApplication()
killApplication()	杀死应用	发送 RMApKillByClientEvent
getApplicationReport()	查询应用状态	rmApp.createAndGetApplicationReport()
getClusterMetrics()	查询集群指标	scheduler.getClusterResource()
moveApplication()	跨队列移动应用	scheduler.moveApplication()

表3-2: ClientRMService 核心RPC方法

3.3 RMApManager

负责应用的提交、完成和清理。核心方法 submitApplication() 创建 RMApImpl 实例, 发送 START 事件触发状态机; finishApplication() 执行收尾; checkAppNumCompletedLimit() 按 LRU 策略淘汰已完成应用, 维护内存和 StateStore 上限。实现 Recoverable 接口, 支持 RM 重启后从 StateStore 恢复应用。

3.4 RMAppImpl

RMAp 的默认实现, 核心是 StateMachineFactory 构建的状态机(详见2.2节):

字段	类型	说明
applicationId	ApplicationId	应用唯一ID(不可变)
submissionContext	ApplicationSubmissionContext	应用提交上下文(含AM启动信息)
attempts	LinkedHashMap	所有尝试实例(按插入顺序)
currentAttempt	volatile RMApAttempt	当前活跃尝试
stateMachine	StateMachine	状态机实例
updatedNodes	Map	待处理的节点更新

表3-3: RMAppImpl 核心字段

3.5 RMNodeImpl

RM 视角的节点抽象, 实现事件驱动的状态机模式(详见2.6节)。维护节点资源(totalCapability/allocatedResource)、已启动容器集合、待清理容器/应用列表、容器更新队列、节点利用率等。StatusUpdateWhenHealthyTransition 处理心跳中的容器状态更新, 并通过 NODE_UPDATE 事件触发 Scheduler 进行资源分配。

第四章 调度器设计

4.1 调度器继承体系

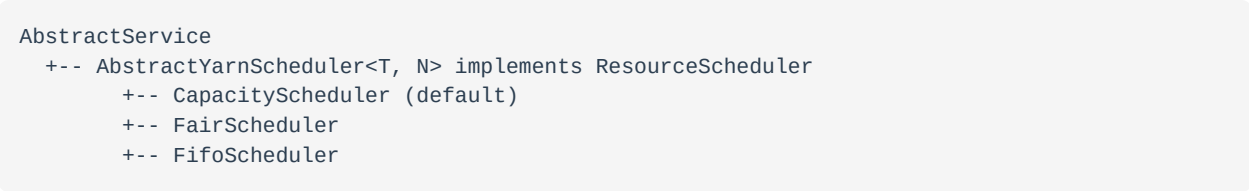


图4-1: 调度器类继承体系

AbstractYarnScheduler 提供公共能力: 集群节点追踪(ClusterNodeTracker)、应用映射表、读写锁、定期更新线程、容器生命周期管理和最小/最大资源配置。

4.2 CapacityScheduler (默认调度器)

YARN 生产环境的默认调度器, 采用树形层级队列实现多租户资源隔离:

特性	说明
层级队列	CSQueue -> AbstractCSQueue -> ParentQueue/LeafQueue 树形结构
容量配置	支持百分比模式和权重(weight)模式; 有 capacity/maxCapacity 两层配置
弹性共享	空闲资源可被其他队列借用, 当自身需要时通过抢占收回
抢占	PreemptionManager + SchedulingMonitor; 支持懒抢占(先标记后回收)
异步调度	AsyncSchedulingConfiguration 配置独立调度线程
多节点放置	multiNodePlacementEnabled, 支持跨节点寻找最优放置
动态队列	ParentQueue.addDynamicLeafQueue() 运行时自动创建
动态配置	MutableCSConfigurationProvider 支持不重启修改队列配置
节点标签	RMNodeLabelsManager 实现节点分区, 队列绑定特定标签

表4-1: CapacityScheduler 核心特性

4.3 FairScheduler

公平调度器通过 update() 方法定期更新: 递归计算各队列资源需求(updateDemand), 基于公平份额算法分配(update), 更新指标。支持数据局部性调度(node/rack locality), 配置文件热加载(AllocationFileLoaderService), 以及抢占(updateStarvedApps)。

4.4 FifoScheduler

最简单的调度器, 只有一个全局默认队列(DEFAULT_QUEUEUE, 容量100%)。使用 ConcurrentSkipListMap 按应用提交顺序排序, 严格 FIFO 分配资源。仅使用 DefaultResourceCalculator(只按内存计算)。适合测试和简单场景。

第五章 NodeManager 核心类设计

5.1 NodeManager 主入口

NodeManager 继承 CompositeService, 是每个集群节点上的代理进程, 通过工厂方法组装以下子服务:

子服务	职责
NMStateStoreService	LevelDB 状态持久化, 支持 NM 重启恢复容器
ContainerExecutor	容器执行器(Default/Linux/Docker), 通过反射创建
DeletionService	异步文件删除服务
NodeHealthCheckerService	节点健康检查(磁盘/脚本)
NodeStatusUpdaterImpl	向 RM 注册节点 + 定期心跳线程
ContainerManagerImpl	容器生命周期管理(启动/监控/本地化/日志聚合)
WebServer	Web UI + REST API (NMWebServices)
AsyncDispatcher	NM 级全局异步事件分发

表5-1: NodeManager 子服务组成

5.2 ContainerManagerImpl

自身也是 CompositeService, 实现 ContainerManagementProtocol。构造函数中注册8种事件处理器:

事件类型	处理器	说明
ContainerEventType	ContainerEventDispatcher	路由到具体Container实例
ApplicationEventType	ApplicationEventDispatcher	路由到NM端Application
LocalizationEventType	ResourceLocalizationService	资源下载本地化
ContainersLauncherEventType	ContainersLauncher	容器启动/清理/信号
ContainerSchedulerEventType	ContainerScheduler	保证型+机会型容器调度
ContainersMonitorEventType	ContainersMonitor	容器资源使用监控
LogHandlerEventType	LogHandler	日志聚合/非聚合处理

表5-2: ContainerManagerImpl 事件注册

5.3 ContainerLaunch 启动流程

ContainerLaunch 实现 Callable 接口, 由 ContainersLauncher 线程池异步执行:

步骤	操作
1	验证容器状态 (validateContainerState)
2	获取已本地化的资源列表
3	变量展开: 替换 LOG_DIR、CLASS_PATH_SEPARATOR 等占位符
4	确定工作目录、日志目录、PID文件路径
5	检查磁盘健康状态
6	写入 keystore/truststore (安全集群)

7	写入容器启动脚本 (launch_container.sh)
8	调用 ContainerExecutor.launchContainer() -- 阻塞直到容器退出
9	根据退出码发送 EXITED_WITH_SUCCESS 或 EXITED_WITH_FAILURE

表5-3: ContainerLaunch 启动步骤

5.4 ContainerImpl 状态机

NM 端容器的核心实现(详见2.4节)。17个状态, 支持资源本地化、暂停/恢复、重初始化、升级回滚和重试策略(ContainerRetryContext)。采用读写锁保护并发, volatile 保证 launchContext 可见性。

5.5 NodeStatusUpdaterImpl 心跳

负责 NM 与 RM 的通信。生命周期: serviceInit(获取节点资源配置) -> serviceStart(创建RPC代理, 注册节点, 启动心跳线程) -> serviceStop(注销节点)。心跳线程循环构建 NodeHeartbeatRequest 并发送; 处理 RM 返回的指令: SHUTDOWN/RESYNC/NORMAL; 更新 MasterKey; 支持 sendOutOfBandHeartBeat() 立即通知。

第六章 协议与客户端 API

6.1 四大协议接口

协议	通信双方	核心方法
ApplicationClientProtocol	Client <-> RM	getNewApplication, submitApplication, forceKillApplication, getApplicationReport, getClusterMetrics, getQueueInfo
ApplicationMasterProtocol	AM <-> RM	registerApplicationMaster, allocate (核心心跳+资源请求), finishApplicationMaster
ContainerManagementProtocol	AM <-> NM	startContainers, stopContainers, getContainerStatuses, updateContainer, reinitializeContainer
ResourceManagerAdministrationProtocol	Admin <-> RM	refreshQueues, refreshNodes, updateNodeResource, addToClusterNodeLabels

表6-1: YARN 四大协议接口

`allocate()` 方法是整个 YARN 资源调度的核心入口，它同时兼具：发送 `ResourceRequest` 列表请求新容器、归还不需要的容器、更新黑名单、作为心跳保持 AM 存活、返回已分配容器和 `headroom` 信息。标注为 `@AtMostOnce` 语义。

6.2 YarnClient

封装 `ApplicationClientProtocol`，继承 `AbstractService`。核心方法：`createApplication()` 获取新ID；`submitApplication()` 提交并轮询确认；`killApplication()` 杀死并轮询；`getApplicationReport()` 查询(支持回退到AHS)。采用异步轮询确认模式，处理 RM HA 故障转移时的自动重提交。

6.3 AMRMClient

封装 `ApplicationMasterProtocol`，核心方法：`registerApplicationMaster()` 注册；`allocate(progressIndicator)` 心跳+资源请求；`unregisterApplicationMaster()` 注销。内含 `ContainerRequest` 内部类封装容器需求(`capability/nodes/racks/priority/executionType`)。本地维护 `RemoteRequestsTable`，批量发送。支持 `ApplicationMasterNotRegisteredException` 时自动重注册。

6.4 NMClient

封装 `ContainerManagementProtocol`，通过 `ContainerManagementProtocolProxy` 管理与多个 NM 的代理连接。核心方法：`startContainer()`，`stopContainer()`，`getContainerStatus()`，`updateContainerResource()`，`reinitializeContainer()`。使用 `ConcurrentMap` 跟踪已启动容器，`synchronized` 防止 start/stop 竞争。默认停止时自动清理所有运行中容器。